

♥ Assignment: ! ♦ A CONCEPTUAL DIVE INTO MERGE SORT AND QUICK SORT

♥ Merge Sort

Definition

Merge Sort is a classic sorting algorithm that works by recursively dividing the main list into two halves until each sub-list contains only one element (which is inherently sorted). It then repeatedly merges these sorted sub-lists back together in the correct order until the entire list is sorted.



VOX. Of like sorting a large pile of exam papers by student ID. You could split the pile in half, give each half to an assistant, and ask them to do the same. This continues until each person has just one paper. Then, you start merging the sorted single papers back into sorted piles of two, then four, and so on. until you have one single, perfectly ☺



Algorithm



The algorithm consists of two main functions: one for recursively splitting the array (MergeSort) and one for merging the sorted halves (Merge).



MergeSort(arr, left, right):

- If left is greaterer than or equal to r: the segment is sorted (base case).
- Return.
- Calculate $mid = (left + right) / 2$.
- Call MergeSort(arr, left, mid).
- Call Merge(arr, left, mid, right).



Merge(arr, left, mid, right):

- Create temporary arrays for left half (L) and right half (R).
- Copy data from arr[left..mid] to L and from arr[mid+1..right] to R.
- After one of a temporary arrays is exhausted, copy the remaining elements from other other array into arr.



Step-by-Step Examples

Example 1: Sort the array
[8, 3, 5, 1, 9, 2]

Step 1: Divide

The array is recursively split until we have lists of size one

[8, 3, 5, 1, 9, 2] $\rightarrow$

Split [8, 3, 5] into [8] and [3, 5]

Split [3, 5] into [3] and [5]

Split [1, 9, 2] into [1] and [9, 2]

Now, we have the base cases:
[8], [3], [3], [2]

Step 2: Merge and Conquer

Now, we merge these sub arrays back together in sorted order.

Merge [3] and [5] $\rightarrow$ Merge [8] and [3, 5] $\rightarrow$ [3, 5]

Merge [1] and [2, 9] $\rightarrow$ Merge [1] and [2, 9] $\rightarrow$ [1, 2, 9]

Finally, merge the two main sorted halves: [3, 5, 8] and [1, 2, 9]

Compare 3 and 1. Take 1. Array: [1]

Compare 3 and 2. Take 2. Array: [1, 2]

Compare 3 and 9. Take 5. Array: [1, 2, 3, 3]

Compare 5 and 9. Take 5. Array: [1, 2, 3, 5, 8, 8]

Left sub-array is empty. Take remaining 9. Array: [1, 2, 3, 5, 8, 9]

Final Sorted Array: [1, 2, 3, 5, 8, 9]

EXAMPLE 2: SORT THE ARRAY [38, 27, 43, 3, 9, 82, 10]!

STEP 1: DIVIDE

[38, 27, 43, 3, 9, 82, 10]

⇒ [38, 27, 43, 3] and [9, 82, 10]

⇒ [38, 27] and [43, 3] [9] and [82]

⇒ [38] [27] [43] [3] [9] [82]

COMPLEXITY
ANALYSIS

STEP 2: MERGE and CONQUER

Merge [38] and [27] → [27, 38]

Merge [43] and [3] → [3, 43]

Merge [27, 38] and [3, 43] → [3, 27, 38, 43]

Merge [82] and [10] → [10, 82]

Finally, [9] and [10, 82] → [9, 10, 82]

Finally, merge [3, 27, 38, 43] and [9, 10, 82]
⇒ [3, 9, 10, 27, 38, 43, 82]

COMPLEXITY ANALYSIS

$O(n)$



Merge Sort requires temporary arrays to store the halves while merging, so its space complexity is linear with the size of the input.

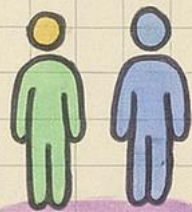
2. Quick Sort!

Definition

Quick Sort is another "Divide and Conquer" algorithm that is often faster in practice than Merge Sort. It works by selecting a single element from the array, called the **pivot**, and rearranging the other elements into two sub-arrays according to whether they are less than or **greater** than the **pivot**. This process is called **partitioning**.



Imagine lining up students by height. You pick one student as the 'pivot' and tell everyone shorter to stand to their left and everyone taller to stand to their right. Now you have two smaller unsorted groups, and you just repeat the same process with each group until everyone is in the correct order.



Algorithm

The algorithm relies on a **Partition** function that places the **pivot** in its correct sorted position. We'll use the common Lomuto partition scheme where the last element is the pivot

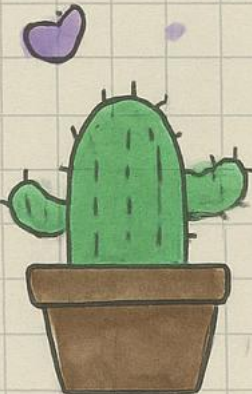
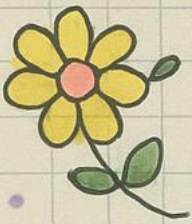
QuickSort(arr, low, high):

If low is less than high: 

- Call **Partition**(arr, low, high) to get the pivot's final index, pi.
- Recursively call **QuickSort**(arr, low, pi - 1) for the left sub-array.
- Recursively call **QuickSort**(arr, pi + 1, high) for the right sub-array.

Partition(arr, low, high):

- Choose the last element (arr[high]) as the pivot.
- Initialize an index **i** (pivot boundary) to low + 1
- iterate with **j** from low to high - 1.
- If arr[j] is less than or equal to the pivot, increment **i** and swap arr[i] with arr[j]
- After the loop, swap the pivot element (arr[high]) with arr[i + 1]
- Return **i + 1** (the pivot's new index).



EXAMPLE 2

Example 2: Sort the array
[10, 80, 30, 90, 50, 70]

Round 1:

low = 0, high = 6, pivot = 70

Initial Array:
[10, 80, 30, 90,
40, 50]

Partitioned Array:
[10, 30, 40, 50, 70, 90, 80]

Pivot 70 is now at index 4.

Round 2 (Left):
low = 0, high = 3,
pivot = 50 on
sub-array [10, 30,]
and
QuickSort(90, 80),
5, 6)

Round 2 (Right):
low = 5, high = 6,
pivot = 80 on
sub-array [90, 80]

Pivotal call:
QuickSort[90], 6, 6:
(base case)

The recursion continues until all partitions
are of size one.

Final Sorted Array:
[10, 30, 40, 50, 50, 70, 80]

COMPARATIVE ANALYSIS: MERGE SORT vs. QUICK SORT

Comparative Analysis:		Quick Sort
Time Complexity	$O(n \log n)$ (All Cases)	Average: $O(n \log n)$, Worst: $O(n^2)$
Space Complexity	Stable	Average: $O(\log n)$, Worst: $O(n)$
Stability ¹	Consistent and predictable.	Often faster in practice due to lower constant factors and cache efficiency.
Worst Case	Never degrades.	Ideal for arrays. The default choice for in-memory sorting in many standard libraries.
Use Case	Ideal for linked lists and when stability is required.	

¹Stability refers to whether the algorithm maintains the relative order of equal elements. Merge Sort does, but Quick Sort's swapping mechanism does not.